

Midterm Project Report

Bryce Poonai

Introduction

Since their introduction in 2018, fine tunable LLMs have become more prevalent and accessible than ever before. Especially due to the capabilities of modern hardware, we are able to access these models and tune them at home either via your local PC or remotely into cloud server hardware. Our goal for this midterm is to finetune a specific LLM model with a given dataset. We also need to participate in a competition where the models will be evaluated and scored amongst other participants. We are also provided with a starter Google Colab Jupyter notebook that contains all the steps for installing the necessary libraries, processing and implementing the data set, configuring the training parameters, running the training process, evaluating the model and generating the csv file used for the competition.

Dataset and Model Description

We are given a dataset that contains 1,000,000 math problems. The questions vary from Numerical questions, Algebra word questions, Trigonometry, etc. Each question has a solution prompt, an answer and a boolean true or false answer. The samples are distributed so that 50% of the questions have solutions that are correct and 50% of the questions have solutions that are incorrect. The LLM model that we are assigned is the Llama-3-8B from Meta. Since the model has already been pretrained, we are tasked to fine tune the model further, so it can better predict the validity of the solutions to the given questions in the dataset. This means that the model will perform better by predicting the right true/false outcome when observing the given question and solution.

Experiments and Results

Since we have limited resources for tweaking and training/testing our model, we need to find a balance between how many parameters we should change before we train/test, and how

much training/testing we should do for both the amount of data and amount of training cycles. The approach that was chosen was training the model using a large dataset of examples within one epoch. First I decided to make the prompt more simplified and increase the amount of training data to 40000 and validation data to 4000 and then train to see the results:

```
# The instructions prompt template for training
training_prompt = """You are tasked with predicting if a solution to a given math question is correct or not. Your response should be 'True' if the solution is correct, otherwise 'False'. Below is the Question and Solution.
Question:
{question}
Solution:
{solution}
Output:
{output}
"""

# We must add an end of sequence (EOS) token to tell the model when a completion is finished.
EOS_TOKEN = tokenizer.eos_token

# This function formats our data samples into the prompt template.
def formatting_prompts_func(examples):
    questions = examples['question']
    solutions = examples['solution']
    outputs = examples['is_correct']
    texts = []
    for question, solution, output in zip(questions, solutions, outputs):
        # Format the prompt and add the EOS token
        text = training_prompt.format(question=question, str(solution), str(output)) + EOS_TOKEN
        texts.append(text)
    return {'text': texts}

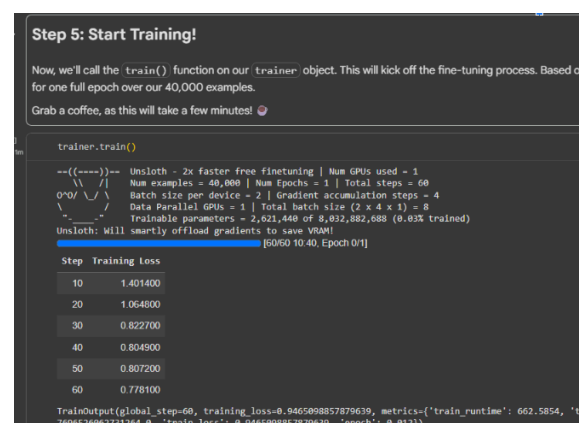
# Apply the formatting function to our training dataset
formatted_train_dataset = train_dataset.map(formatting_prompts_func, batched=True)
```

```
from datasets import load_dataset

# Load the full training dataset
full_dataset = load_dataset("ad6398/nyu-dl-teach-maths-comp", split="train")

# Shuffle the dataset for randomness and create our smaller splits
shuffled_dataset = full_dataset.shuffle(seed=42)
train_dataset = shuffled_dataset.select(range(40000)) # Use the first 40,000 for training
validation_dataset = shuffled_dataset.select(range(40000, 44000)) # Use the next 4,000 for validation
```

Without changing any of the LoRA and SFT Trainer settings, these are how the training loss values look:



Next we need to do a little research for optimizing our LoRA configuration. According to the Unsloth LoRA optimization guide¹, it would make sense to use a rank of either 16 or 32¹. I decided to use 16 because it gives us a good balance of fast tuning with less risk of overfitting. We would then need to set alpha to $2 * r$ as recommended by the document. For the rest of the parameters, they can be left as they are for now as they are left at the recommended settings.

```
model = FastLanguageModel.get_peft_model(
    model,
    r = 16, # Regular rank
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                     "gate_proj", "up_proj", "down_proj"],
    lora_alpha = 32, # A common practice is to set alpha = 2 * r
    lora_dropout = 0,
    bias = "none",
    use_gradient_checkpointing = "unsloth",
    random_state = 42,
)
```

Now let's rerun the code with these tweaked parameters, and see what the training losses look like:

```
trainer.train()

==((====))== Unsloth - 2x faster free finetuning | Num GPUs used = 1
              \  / Num examples = 40,000 | Num Epochs = 1 | Total steps = 60
0'0' / \ / \ / Batch size per device = 2 | Gradient accumulation steps = 4
              \ / Data Parallel GPUs = 1 | Total batch size (2 x 4 x 1) = 8
              - - - - - Trainable parameters = 41,943,040 of 8,072,204,288 (0.52% trained)
Unsloth: Will smartly offload gradients to save VRAM! [60/60 10:23, Epoch 0/1]

Step Training Loss
10 1.142300
20 0.793500
30 0.719100
40 0.742700
50 0.755000
60 0.723700

TrainOutput(global_step=60, training_loss=0.8127192656199137, metrics={'train_runtime': 773.6837466046464, 'train_loss': 0.8127192656199137, 'epoch': 0.012})
```

The losses look better than before when we had the much lighter rank and alpha settings. Now we need to change our SFT trainer settings to see the most amount of improvement, but first we need to optimize the batch size and gradient accumulation steps. We will first try the recommended GPU settings on the hugging face GPU guide which are that batch size = 4 and gradient_accumulation_steps = 16². We also need to change our max steps size to accommodate our training set. The formula for max steps size (according to google) is: $\text{max_steps} = \frac{\text{train_dataset}}{(\text{batch_size} * \text{gradient_accumulation_steps})} * \text{num_train_epochs} = \frac{40000}{(4 * 16)} * 3 = 1875$

```
from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = formatted_train_dataset,
    dataset_text_field = "text",
    max_seq_length = max_seq_length,
    args = TrainingArguments(
        per_device_train_batch_size = 4,
        gradient_accumulation_steps = 16,
        num_train_epochs = 3,
        warmup_steps = 5,
        max_steps = 1875,
        learning_rate = 2e-4,
        fp16 = not torch.cuda.is_bf16_supported(),
        bf16 = torch.cuda.is_bf16_supported(),
        logging_steps = 10,
        optim = "adamw_8bit",
        weight_decay = 0.01,
        lr_scheduler_type = "linear",
        seed = 42,
        output_dir = "outputs",
        report_to = "none",
    ),
)
```

After testing these settings we were able to use 9.4/15GB of VRAM on the GPU, but the training was taking too long. We need to decrease the number of gradient accumulation steps and increase batch size. Trying 8/8 resulted in higher VRAM usage peaking at 14.7/15GB, but training is still very slow. We need to decrease our total batch size. Leaving batch size at 4 makes sense as it'll effectively utilize VRAM without spilling over to DRAM. We just need to tune our gradient accumulation size. Trying 4/4 is faster, but still not fast enough for our large dataset. We should reduce the amount of training data in our dataset. Let's try 20000. $\text{max_steps} = \frac{20000}{(4 * 4)} * 1 = 1250$ This would be a perfect compromise for our training rate with its large number of examples.

Now we need to tune our SFTTrainer parameters. We need some baseline parameters that will give us a good foundation which we can further optimize later. According to the article "Training Arguments of SFT of LLM" by Sivanarayana Mamidi, Mamidi explains how each of the parameters work and what are the recommended values³. The first thing to change is the warmup steps. Instead of using a fixed number of steps, we should use a ratio³. Next the adam beta parameters should be defined, so they can be tuned later. Then we should start with a lower weight decay for our large dataset of 0.0001. For the learning rate scheduler, the cosine curve would most likely give the best results when tuned appropriately³.

```

from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = formatted_train_dataset,
    dataset_text_field = "text",
    max_seq_length = max_seq_length,
    args = TrainingArguments(
        per_device_train_batch_size = 4,
        gradient_accumulation_steps = 4,
        num_train_epochs = 1,
        warmup_ratio = 0.1,
        max_steps = 1250,
        learning_rate = 2e-4,
        fp16 = not torch.cuda.is_bf16_supported(),
        bf16 = torch.cuda.is_bf16_supported(),
        logging_steps = 10,
        optim = "adamw_8bit",
        adam_beta1 = 0.9,
        adam_beta2 = 0.999,
        weight_decay = 0.0001,
        lr_scheduler_type = "cosine",
        seed = 42,
        output_dir = "outputs",
        report_to = "none",
    ),
)

```

If we're trying to tune this model with this step size, it would just take too long, so using a step size of 150 will give us a good idea of how the model is performing while allowing us to tune the model for every run. Here are the results of the first run:

```

trainer.train()

Unsloth - 2x faster free finetuning | Num GPUs used = 1
Num examples = 20,000 | Num Epochs = 1 | Total steps = 150
Batch size per device = 4 | Gradient accumulation steps = 4
Data Parallel GPUs = 1 | Total batch size (4 x 4 x 1) = 16
Trainable parameters = 41,943,040 of 8,072,204,288 (0.52% trained)
[150/150 57.24, Epoch 0/1]

Step Training Loss
10 0.802300
20 0.716700
30 0.738000
40 0.751100
50 0.770100
60 0.728100
70 0.733200
80 0.688900
90 0.711500
100 0.704500
110 0.718900
120 0.737900
130 0.702500
140 0.702900
150 0.710300

TrainOutput(global_step=150, training_loss=0.727793366142985, metrics={'train_runtime': 346.4663801233673421e+16, 'train_loss': 0.727793366142985, 'epoch': 0.12})

```

The error seems to converge around 0.70 loss. Let's tune this further.

The next thing I wanted to tune is the learning rate. Looking at the German version of Llama 3-8B on the hugging face website, they were using a learning rate range of $1.5 \cdot 10^{-5}$ - $1.5 \cdot 10^{-6}$ and a weight decay of 0.05 with the cosine curve⁴. Let's see if they improve our loss rate for our model. I will choose $1 \cdot 10^{-5}$ with the 0.05 weight decay.

```

trainer.train()

Unsloth - 2x faster free finetuning | Num GPUs used = 1
Num examples = 20,000 | Num Epochs = 1 | Total steps = 150
Batch size per device = 4 | Gradient accumulation steps = 4
Data Parallel GPUs = 1 | Total batch size (4 x 4 x 1) = 16
Trainable parameters = 41,943,040 of 8,072,204,288 (0.52% trained)
Unsloth: Will smartly offload gradients to save VRAM!
[150/150 52.11, Epoch 0/1]

Step Training Loss
10 1.463300
20 1.415300
30 1.307300
40 1.142000
50 1.013200
60 0.904800
70 0.865200
80 0.803200
90 0.816600
100 0.816500
110 0.822000
120 0.840400
130 0.791100
140 0.804000
150 0.807600

TrainOutput(global_step=150, training_loss=0.9741589228312174, metrics={'train_runtime': 3158.11154.663801233673421e+16, 'train_loss': 0.9741589228312174, 'epoch': 0.12})

```

The training losses are higher, but we can't really gauge how the model will perform until we train it for one full epoch and test it, so let's fully train this model by increasing the number of steps.

```

trainer.train()

950 0.711000
960 0.705200
970 0.726500
980 0.708400
990 0.707100
1000 0.741800
1010 0.664100
1020 0.732600
1030 0.708400
1040 0.713800
1050 0.689500
1060 0.721600
1070 0.694200
1080 0.690700
1090 0.678400
1100 0.708400
1110 0.722700
1120 0.722900
1130 0.721500
1140 0.708000
1150 0.707700
1160 0.709200
1170 0.726900
1180 0.704300
1190 0.673200
1200 0.717000
1210 0.702900
1220 0.692200
1230 0.718700
1240 0.734500
1250 0.720100

TrainOutput(global_step=1250, training_loss=0.76972790408476553, metrics={'train_runtime': 346.4663801233673421e+16, 'train_loss': 0.76972790408476553, 'epoch': 0.12})

```

This is how this model performed in training. The training losses are only slightly better compared to the starter parameters. This was the first submission, and we were met with a score of around 0.55 which is significantly worse than the starting csv file. When trying a lower learning

rate of $1.5e^{-6}$ we are met with worse training losses.

```
from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = formatted_train_dataset,
    dataset_text_field = "text",
    max_seq_length = max_seq_length,
    args = TrainingArguments(
        per_device_train_batch_size = 4,
        gradient_accumulation_steps = 4,
        num_train_epochs = 1,
        warmup_ratio = 0.1,
        max_steps = 1250,
        learning_rate = 1.5e-6,
        fp16 = not torch.cuda.is_bf16_supported(),
        bf16 = torch.cuda.is_bf16_supported(),
        logging_steps = 10,
        optim = "adamw_8bit",
        adam_beta1 = 0.9,
        adam_beta2 = 0.999,
        weight_decay = 0.001,
        lr_scheduler_type = "cosine",
        seed = 42,
        output_dir = "outputs",
        report_to = "none",
    ),
)
```

```
trainer.train()

960 0.767800
970 0.790300
980 0.772700
990 0.768200
1000 0.809200
1010 0.728700
1020 0.794400
1030 0.772000
1040 0.776300
1050 0.752100
1060 0.785800
1070 0.760200
1080 0.754200
1090 0.749500
1100 0.772200
1110 0.787800
1120 0.784300
1130 0.786800
1140 0.775400
1150 0.768000
1160 0.773300
1170 0.791800
1180 0.767100
1190 0.737600
1200 0.775300
1210 0.768100
1220 0.756200
1230 0.782400
1240 0.797200
1250 0.784300

TrainOutput(global_step=1250, training_loss=0.8913529586791992, metrics={'train_runtime': 29343.7334,
3.859027410842419e+17, 'train_loss': 0.8913529586791992, 'epoch': 1.0})
```

The training losses were higher and the submission score lowered to 0.44. We haven't run the linear scheduler yet, so let's try running the trainer with the same parameters with the linear scheduler and see where it converges.

```
from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = formatted_train_dataset,
    dataset_text_field = "text",
    max_seq_length = max_seq_length,
    args = TrainingArguments(
        per_device_train_batch_size = 4,
        gradient_accumulation_steps = 4,
        num_train_epochs = 1,
        warmup_ratio = 0.1,
        learning_rate = 1.5e-6,
        fp16 = not torch.cuda.is_bf16_supported(),
        bf16 = torch.cuda.is_bf16_supported(),
        logging_steps = 10,
        optim = "adamw_8bit",
        adam_beta1 = 0.9,
        adam_beta2 = 0.999,
        weight_decay = 0.01,
        lr_scheduler_type = "linear",
        seed = 42,
        output_dir = "outputs",
        report_to = "none",
    ),
)
```

```
960 0.769500
970 0.792100
980 0.774400
990 0.769700
1000 0.810700
1010 0.730100
1020 0.795600
1030 0.773000
1040 0.777300
1050 0.752900
1060 0.786600
1070 0.760900
1080 0.754800
1090 0.746100
1100 0.772600
1110 0.788000
1120 0.784500
1130 0.787000
1140 0.775300
1150 0.767900
1160 0.773200
1170 0.791600
1180 0.766800
1190 0.737200
1200 0.774800
1210 0.767600
1220 0.755700
1230 0.781800
1240 0.796700
1250 0.783800

TrainOutput(global_step=1250, training_loss=0.8950863330841065, metrics={'train_runtime': 25
'train steps per second': 0.049, 'total flops': 3.859027410842419e+17, 'train_loss': 0.895086
```

There isn't much of a difference between the two LR schedulers when looking at the training losses, but the cosine model seems to be the better choice with the more dynamic learning rate fluctuations.

Next, I want to try to run for 2 epochs with a higher weight decay value to prevent overfitting, but the runtime occasionally disconnects since the training duration is around 16 hours.

```
1590 0.711100
1600 0.732700
1610 0.737300
1620 0.739100
1630 0.750700
1640 0.746900
1650 0.729200
1660 0.735700
1670 0.778800
1680 0.742600
1690 0.731800
1700 0.747100
1710 0.730600
1720 0.716200
1730 0.728100
1740 0.732700
1750 0.735500
1760 0.743600
1770 0.717700
1780 0.781500
1790 0.717000
1800 0.756000
1810 0.733000
1820 0.779000
1830 0.717000
1840 0.736200
1850 0.738400
1860 0.726900
1870 0.708200
```

The results seem to be improving with the 2nd epoch. So, training the model using many examples within one epoch doesn't seem to be optimal due to the drastically longer training times and the training losses weren't improving. There could've been more testing to be done to improve the performance of this further by increasing the learning rate, but the deadline for the submissions were close. Trying 5 epochs with a higher learning rate nets us these results:

```
from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = formatted_train_dataset,
    dataset_text_field = "text",
    max_seq_length = max_seq_length,
    args = TrainingArguments(
        per_device_train_batch_size = 4,
        gradient_accumulation_steps = 4,
        num_train_epochs = 5,
        warmup_ratio = 0.1,
        learning_rate = 5e-5,
        fp16 = not torch.cuda.is_bf16_supported(),
        bf16 = torch.cuda.is_bf16_supported(),
        logging_steps = 10,
        optim = "adamw_8bit",
        weight_decay = 0.001,
        lr_scheduler_type = "linear",
        seed = 42,
        output_dir = "outputs",
        report_to = "none",
    ),
)
```

320	0.634500
330	0.626900
340	0.615300
350	0.606500
360	0.634800
370	0.629000
380	0.604600
390	0.576500
400	0.559200
410	0.556600
420	0.571200
430	0.556000
440	0.561500
450	0.558900
460	0.601800
470	0.588500
480	0.576100
490	0.554700
500	0.559400
510	0.524900
520	0.530300
530	0.514500
540	0.523400
550	0.490000
560	0.506300
570	0.496500
580	0.493300
590	0.518600
600	0.502500
610	0.505400
620	0.522900

TrainOutput(global_step=625, training_loss=0.6608666919708251, metrics={'train_runtime':

```
from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = formatted_train_dataset,
    dataset_text_field = "text",
    max_seq_length = max_seq_length,
    args = TrainingArguments(
        per_device_train_batch_size = 4,
        gradient_accumulation_steps = 4,
        num_train_epochs = 5,
        warmup_ratio = 0.1,
        learning_rate = 5e-5,
        fp16 = not torch.cuda.is_bf16_supported(),
        bf16 = torch.cuda.is_bf16_supported(),
        logging_steps = 10,
        optim = "adamw_8bit",
        weight_decay = 0.0001,
        lr_scheduler_type = "cosine",
        seed = 42,
        output_dir = "outputs",
        report_to = "none",
    ),
)
```

330	0.622400
340	0.610500
350	0.601400
360	0.629800
370	0.623900
380	0.598100
390	0.569500
400	0.551300
410	0.548600
420	0.562300
430	0.547400
440	0.551700
450	0.549700
460	0.592700
470	0.578300
480	0.567100
490	0.544700
500	0.549600
510	0.527100
520	0.533200
530	0.517200
540	0.526400
550	0.493600
560	0.508200
570	0.502200
580	0.499000
590	0.521700
600	0.506200
610	0.509200
620	0.526900

TrainOutput(global_step=625, training_loss=0.6612388999938965, metrics={'train_runtime': 1.902582255178875e+17, 'train_loss': 0.6612388999938965, 'epoch': 5.0})

The results are much better with the higher learning rate with 5 epochs. Both these models scored a 78 on the leaderboard. The cosine model with more examples was the best model so far. The next test is to increase the learning rate and run the model. Here are the results of that:

```

from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = formatted_train_dataset,
    dataset_text_field = "text",
    max_seq_length = max_seq_length,
    args = TrainingArguments(
        per_device_train_batch_size = 4,
        gradient_accumulation_steps = 4,
        num_train_epochs = 5,
        warmup_ratio = 0.1,
        learning_rate = 9e-5,
        fp16 = not torch.cuda.is_bf16_supported(),
        bf16 = torch.cuda.is_bf16_supported(),
        logging_steps = 10,
        optim = "adamw_8bit",
        weight_decay = 0.0001,
        lr_scheduler_type = "cosine",
        seed = 42,
        output_dir = "outputs",
        report_to = "none",
    ),
)

```

400	0.567100
410	0.541900
420	0.531100
430	0.461600
440	0.454700
450	0.445500
460	0.451700
470	0.465900
480	0.459700
490	0.469700
500	0.461500
510	0.447800
520	0.463900
530	0.470000
540	0.436500
550	0.452800
560	0.397700
570	0.417200
580	0.398900
590	0.406400
600	0.380300
610	0.373800
620	0.358600
630	0.374000
640	0.355400
650	0.373100
660	0.381900
670	0.386000
680	0.370400
690	0.383800

The training losses look way better than before. Unfortunately, as the csv file was almost about to finish generating, I ran out of compute units, and the runtime was on a time limit and disconnected. This was also right before the competition was about to end, so there wasn't any more time to rerun everything on a different client.

Conclusion

The approach that was chosen wasn't ideal for this project. Using one epoch with many examples took too long to train the model and the results were not great. This resulted in a lot of time and compute units being used up, and then when trying a different approach, there wasn't much time and compute units to spare for optimizing the model. However, this project was a great learning experience. It really emphasized the differences between training and fine tuning and allowed for experimentation with many approaches which can possibly work or possibly not work. Looking back at the beginning of the project, it would've been better to just experiment with multiple approaches instead of trying to make one approach work. It's also really unfortunate to be close to having a potentially better model that could've given better results than the chosen submissions but then be cut short because of running out of compute units. Having another partner would've been beneficial for having more resources, but regardless, there is a slight improvement in the score with this model compared to the starter model parameters.

References

- [1] Eyera, & Unsloth Writing Team. (2025, June 28). Lora Hyperparameters Guide. Unsloth. <https://docs.unsloth.ai/get-started/fine-tuning-llm-s-guide/lora-hyperparameters-guide>
- [2] Hugging Face Writing Team. (n.d.). GPU. Hugging Face. https://huggingface.co/docs/transformers/en/perf_train_gpu_one?mixed-precision=bf16
- [3] Mamidi, S. (2024, August 31). Training arguments of SFT of LL. Medium. <https://medium.com/@sivamamidi1998/training-arguments-of-sft-of-ll-ca2732599623>
- [4] Hugging Face Writing Team. (n.d.-b). Llama3-German-8B (version 0.1). Hugging Face. <https://huggingface.co/DiscoResearch/Llama3-German-8B>

Github and Model Links

<https://github.com/bwp7162/ECE-7123-Midterm-Project-Repo/tree/main>

<https://colab.research.google.com/drive/182olYkPsW71iNRBwNR2V2hv9YskEndYD?usp=sharing>

